

EE708: Assignment 6

Chaitanya Vishnoi (230316)

Q1. Logistic Regression vs Perceptron

Perceptron Model

$$y = \text{sign}(w^T x + b)$$

Uses a step function, which is not differentiable, hence gradient-based optimization cannot be applied.

Logistic Regression Model

$$y = \sigma(z) = \frac{1}{1+e^{-z}}, \quad z = w^T x + b$$

Loss Function (Cross-Entropy)

$$E(w) = - \sum_{i=1}^N [t_i \log y_i + (1 - t_i) \log(1 - y_i)]$$

Interpretation:

- If $t_i = 1$: $E = -\log(y_i)$, penalizes low predicted probability
- If $t_i = 0$: $E = -\log(1 - y_i)$, This is cross-entropy loss

Gradient-Based Optimization

Because sigmoid is differentiable:

$$\frac{\partial E}{\partial w} = (y - t)x$$

This enables gradient descent and backpropagation in logistic regression which is not possible in perceptron.

Update Rule:

$$w \leftarrow w - \eta(y - t)x$$

Comparison

Perceptron: $w \leftarrow w + \eta tx$

Logistic: $w \leftarrow w - \eta(y - t)x$

Key Difference is loss:

- Perceptron: Misclassification loss
- Logistic: Smooth cross-entropy loss

Conclusion

Logistic regression is preferred because:

- Smooth optimization landscape
- Probabilistic output
- Works for non-separable data
- Perceptron fails if data not linearly separable
- Logistic minimizes cross-entropy loss

If we change the activation function from signum to sigmoid, the output changes from a binary to a probabilistic one, this completely changes a single perceptron model to logistic regression.

Q2. Output Layer Design

Binary Classification (Spam vs Ham)

- Output: 1 neuron
- Activation: Sigmoid

Sigmoid activation function is used for binary classification cases because :

$$\sigma(z) \in (0, 1)$$

Interpreted as:

$$P(\text{spam} \mid x)$$

Decision rule:

$$y = \begin{cases} 1 & \text{if } y > 0.5, \\ 0 & \text{otherwise} \end{cases}$$

Multiclass Classification (MNIST)

- Output: 10 neurons
- Activation: Softmax

$$y_i = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}$$
$$\sum_{i=1}^{10} y_i = 1$$

Interpretation:

$$P(y = i \mid x)$$

This produces probability distribution over classes.

Key Difference

Sigmoid	Softmax
Binary	Multi-class
Independent outputs	Coupled outputs
Sum $\neq 1$	Sum = 1

—

Q3. MLP Shapes and Equation

Given,

- Input layer = 10
- Hidden layer = 50
- Output = 3
- Samples = m

Input Matrix :

$$X \in \mathbb{R}^{m \times 10}$$

Hidden Layer :

$$W_h \in \mathbb{R}^{10 \times 50}, \quad b_h \in \mathbb{R}^{1 \times 50}$$

Output Layer :

$$W_o \in \mathbb{R}^{50 \times 3}, \quad b_o \in \mathbb{R}^{1 \times 3}$$

Output Matrix :

$$Y \in \mathbb{R}^{m \times 3}$$

From lecture notes we have,

$$z^{(l)} = W^{(l)} * a^{(l-1)} + b^{(l)}$$

- $l \rightarrow$ current layer index
- $a^{(l-1)} \rightarrow$ activations from previous layer (input to this layer)
- $W^{(l)} \rightarrow$ weight matrix of layer l
- $b^{(l)} \rightarrow$ bias vector of layer l
- $z^{(l)} \rightarrow$ pre-activation (linear output)

Forward Pass

$$H = \text{ReLU}(XW_h + b_h)$$

$$Y = \text{ReLU}(HW_o + b_o)$$

Final Expression

$$Y = \text{ReLU}(\text{ReLU}(XW_h + b_h)W_o + b_o)$$

—

Q4. Cross-Entropy Derivative

Given,

$$E = -[t_k \ln y_k + (1 - t_k) \ln(1 - y_k)]$$

Chain Rule

$$\frac{\partial E}{\partial a_k} = \frac{\partial E}{\partial y_k} \cdot \frac{\partial y_k}{\partial a_k}$$

Derivatives

(i) Cross-Entropy Loss Derivate :

$$\frac{\partial E}{\partial y_k} = -\frac{t_k}{y_k} + \frac{1 - t_k}{1 - y_k}$$

(ii) Sigmoid Derivate :

$$\frac{\partial y_k}{\partial a_k} = y_k(1 - y_k)$$

Multiplication

$$\frac{\partial E}{\partial a_k} = \left(-\frac{t_k}{y_k} + \frac{1 - t_k}{1 - y_k} \right) y_k(1 - y_k)$$

Simplification

$$= -t_k(1 - y_k) + (1 - t_k)y_k$$

$$= -t_k + t_k y_k + y_k - t_k y_k$$

$$= y_k - t_k$$

$$\boxed{\frac{\partial E}{\partial a_k} = y_k - t_k}$$

Conclusion

Cross-entropy cancels sigmoid derivative $\rightarrow$ avoids vanishing gradients

Vanishing gradients break the training hence this derivate makes the training stable.

Q5. Convolution

2	5	-3	0
0	6	0	-4
-1	-3	0	2
5	0	0	3

*

-2	0
4	6

Formula

$$C_{ij} = \sum_{u=0}^1 \sum_{v=0}^1 A_{i+u, j+v} \cdot F_{u,v}$$

This is the mathematical definition of 2D discrete convolution. We are sliding a small filter (kernel) over a larger input matrix.

At each position:

- Take a local patch of the input
- Multiply it element-wise with the filter
- Sum everything this gives one output value

Example

$$C_{11} = (2)(-2) + (5)(0) + (0)(4) + (6)(6) = 32$$

$$C_{12} = (5)(-2) + (-3)(0) + (6)(4) + (0)(6) = 14$$

Final Output

$$\begin{bmatrix} 32 & 14 & -18 \\ -22 & -24 & 12 \\ 22 & 6 & 18 \end{bmatrix}$$

Q6. Strided and Transposed Convolution

Strided Convolution

Activations:

$$x = (x_1, x_2, x_3, x_4)^T$$

Filter:

$$(w_1, w_2, w_3)$$

Stride:

$$2$$

This is the input after padding,

$$x = (0, x_1, x_2, x_3, x_4, 0)^T$$

Output after convolution:

$$y = \begin{bmatrix} w_1 \cdot 0 + w_2 x_1 + w_3 x_2 \\ w_1 x_2 + w_2 x_3 + w_3 x_4 \end{bmatrix}$$

Hence, matrix representation will be as follows

$$y = Ax \implies A = \begin{bmatrix} w_1 & w_2 & w_3 & 0 & 0 & 0 \\ 0 & 0 & w_1 & w_2 & w_3 & 0 \end{bmatrix}, \quad \text{where } x \in \mathbb{R}^6 \text{ and } z \in \mathbb{R}^2$$

Transposed Convolution

Activations :

$$(z_1, z_2)$$

Transpose of Matrix A:

$$A^T = \begin{bmatrix} w_1 & 0 \\ w_2 & 0 \\ w_3 & w_1 \\ 0 & w_2 \\ 0 & w_3 \\ 0 & 0 \end{bmatrix}$$

$$y = A^T z$$

Hence,

$$y = \begin{bmatrix} w_1 z_1 \\ w_2 z_1 \\ w_3 z_1 + w_1 z_2 \\ w_2 z_2 \\ w_3 z_2 \\ 0 \end{bmatrix}$$

—

Q7. Autoencoder (Complete Numerical Solution)

Given

Let input:

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Encoder weights:

$$W_{enc} = \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix}$$

Bottleneck weights:

$$W_{bottle} = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$$

Decoder weights:

$$W_{dec} = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}$$

Output weights:

$$W_{out} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$$

Target output:

$$Y' = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

—

Step 1: Encoder (Matrix Multiplication)

$$H_{pre} = W_{enc}X = \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$H_{pre} = \begin{bmatrix} (1)(1) + (-1)(3) & (1)(2) + (-1)(4) \\ (0)(1) + (2)(3) & (0)(2) + (2)(4) \end{bmatrix} = \begin{bmatrix} -2 & -2 \\ 6 & 8 \end{bmatrix}$$

—

Step 2: Apply ReLU

$$H = \max(0, H_{pre}) = \begin{bmatrix} 0 & 0 \\ 6 & 8 \end{bmatrix}$$

Negative values set to zero explicitly

—

Step 3: Bottleneck

$$Z_{pre} = W_{bottle}H = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 \\ 6 & 8 \end{bmatrix}$$
$$Z_{pre} = \begin{bmatrix} (1)(0) + (1)(6) & (1)(0) + (1)(8) \\ (-1)(0) + (1)(6) & (-1)(0) + (1)(8) \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 6 & 8 \end{bmatrix}$$
$$Z = \max(0, Z_{pre}) = \begin{bmatrix} 6 & 8 \\ 6 & 8 \end{bmatrix}$$

Step 4: Decoder

$$D = W_{dec}Z = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 6 & 8 \\ 6 & 8 \end{bmatrix}$$
$$D = \begin{bmatrix} (1)(6) + (0)(6) & (1)(8) + (0)(8) \\ (2)(6) + (1)(6) & (2)(8) + (1)(8) \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 18 & 24 \end{bmatrix}$$

Step 5: Reconstruction

$$Y = W_{out}D = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 6 & 8 \\ 18 & 24 \end{bmatrix}$$
$$Y = \begin{bmatrix} (1)(6) + (2)(18) & (1)(8) + (2)(24) \\ (0)(6) + (1)(18) & (0)(8) + (1)(24) \end{bmatrix} = \begin{bmatrix} 42 & 56 \\ 18 & 24 \end{bmatrix}$$

Step 6: Loss Calculation (MSE)

$$(Y - Y') = \begin{bmatrix} 42 - 1 & 56 - 2 \\ 18 - 3 & 24 - 4 \end{bmatrix} = \begin{bmatrix} 41 & 54 \\ 15 & 20 \end{bmatrix}$$
$$(Y - Y')^2 = \begin{bmatrix} 1681 & 2916 \\ 225 & 400 \end{bmatrix}$$
$$L = \frac{1}{4}(1681 + 2916 + 225 + 400)$$

$$\boxed{L = 1305.5}$$

Step 7: Gradient Derivation

$$L = \frac{1}{n} \sum (Y - Y')^2$$

Differentiate:

$$\frac{\partial L}{\partial Y} = \frac{2}{n} (Y - Y')$$

$$\frac{2}{4} \begin{bmatrix} 41 & 54 \\ 15 & 20 \end{bmatrix} = \begin{bmatrix} 20.5 & 27 \\ 7.5 & 10 \end{bmatrix}$$

Step 8: Backpropagation

$$\frac{\partial L}{\partial W_{out}} = \frac{\partial L}{\partial Y} \cdot D^T$$

$$\frac{\partial L}{\partial D} = W_{out}^T \frac{\partial L}{\partial Y}$$

$$\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial D} \odot \text{ReLU}'$$

Conclusion

This shows that Autoencoder minimises :

$$||x - x'||^2$$

The bottleneck layer enforces:

- Feature Extraction
- Dimensionality reduction

This completes full forward and backward computation with explicit values.